Optimized Minhash for Plagiarism Detection

Neha Ramachandra, Jasper Shen,
Neerja Akruwala

Motivation/Problem

- Potential plagiarism or duplicated reporting can be hard to identify across large news corpora.
- Traditional similarity comparisons can become expensive as the number of documents grows.
- This motivates exploring scalable approximate methods for text similarity detection.

Objective

- Implement and optimize MinHash using Numba for document similarity detection.
- Evaluate tradeoffs between runtime efficiency and approximation accuracy.
- Assess whether MinHash can support scalable plagiarism style detection in news articles.

Research Questions

1. How much speedup does a Numba-optimized MinHash implementation achieve compared to exact Jaccard similarity for large-text plagiarism detection?

2. How accurately does Minhash approximate Jaccard similarity across news articles?

3. Can MinHash effectively identify highly similar/ potentially plagiarized news articles within large datasets?

4. Do articles within the same category exhibit higher similarity than randomly selected articles?

Method Overview

Similarity Detection Pipeline

Cleaning + Shingling

Minhash Signatures

Estimated Similarity with Minhash

Compare against Exact Jaccard

Evaluate Runtime, Error, and Category Similarity

Data Loading and Cleaning

Scrape News Corpora Text

- Extracts article text from source URLs
- Filters invalid or inaccessible pages
- Builds enriched dataset with text content

```python
def extract_article_text(html: str) -> str:
    soup = BeautifulSoup(html, "html.parser")

    for tag in soup(["script", "style", "noscript", "iframe", "svg", "form"]):
        tag.decompose()

    candidates = []
    for selector in ("article", "main", "body"):
        node = soup.find(selector)
        if node is not None:
            text = " ".join(node.stripped_strings)
            if text:
                candidates.append(text)

    if not candidates:
        candidates.append(" ".join(soup.stripped_strings))

    best_text = max(candidates, key = len, default = "")
    best_text = re.sub(r"\s+", " ", best_text).strip()
    return best_text
```

scrape_news_corpora.py

Data Loading and Cleaning

Data Loader

- Loads UCI News Aggregator dataset
- Applies preprocessing to full article text
- Supports filtering by category

```python
def _preprocess_text(self, text: str) -> str:
    """Normalize and clean article text for downstream similarity tasks."""
    cleaned = str(text).lower()

    # Remove common dateline prefixes like: "city, state (publisher) -"
    cleaned = self._DATELINE_PATTERN.sub("", cleaned)

    for pattern in self._BOILERPLATE_PATTERNS:
        cleaned = pattern.sub(" ", cleaned)

    # Remove non-alphanumeric characters but keep internal hyphens in words.
    cleaned = re.sub(r"[^a-z0-9\s-]", " ", cleaned)
    cleaned = re.sub(r"(?<!\w)-|-(?!\w)", " ", cleaned)

    if self.remove_stopwords:
        tokens = [tok for tok in cleaned.split() if tok not in self._STOPWORDS]
        cleaned = " ".join(tokens)

    # Collapse tabs/newlines/multiple spaces.
    cleaned = re.sub(r"\s+", " ", cleaned).strip()
    return cleaned
```

data_loader.py

Data Loading and Cleaning

Beginning of Similarity Calculation Code (Cleaning)

- Normalizes and tokenizes article text
- Standardizes document inputs for consistency
- Prepares documents for similarity computation

```python
@staticmethod
def normalize_text(text: str) -> str:
    text = text.lower()
    tokens = _WORD_RE.findall(text)
    return " ".join(tokens)
```

minhash.py

Minhash Implementation

K shingle construction

- Convert each article into overlapping shingles
- Represent documents as hashed sets for comparison

```python
def text_to_shingle_hashes(self, text: str) -> np.ndarray:
    normalized = self.normalize_text(text)
    words = normalized.split()

    if len(words) < self.shingle_size:
        return np.empty(0, dtype=np.uint64)

    shingles = set()
    for i in range(len(words) - self.shingle_size + 1):
        shingle = " ".join(words[i : i + self.shingle_size])
        shingles.add(self._stable_uint64_hash(shingle))

    if not shingles:
        return np.empty(0, dtype=np.uint64)

    return np.array(sorted(shingles), dtype=np.uint64)
```

Minhash Implementation

Minhash Signature Generation

- Generate compressed document signatures using randomized hash functions
- Estimate Jaccard similarity from watching signature values

Numba Optimization

- Accelerate signature computation with compiled numerical code
- Improve runtime performance for large scale comparisons

```python
@njit(cache=True)
def _compute_signature_numba(
    shingles: np.ndarray,
    coeff_a: np.ndarray,
    coeff_b: np.ndarray,
    prime: np.uint64,
) -> np.ndarray:
    """Compute one MinHash signature from a 1D array of shingle hashes."""
    num_hashes = coeff_a.shape[0]
    signature = np.empty(num_hashes, dtype=np.uint64)

    for i in range(num_hashes):
        signature[i] = _MAX_HASH

    if shingles.shape[0] == 0:
        return signature

    for i in range(num_hashes):
        a = coeff_a[i]
        b = coeff_b[i]
        current_min = _MAX_HASH

        for j in range(shingles.shape[0]):
            value = (a * shingles[j] + b) % prime
            if value < current_min:
                current_min = value

        signature[i] = current_min

    return signature
```

Computing Exact Jaccard

```python
def compare_two_texts(self, left_text: str, right_text: str) -> PairSimilarityResult:
    left_signature = self.signature_from_text(left_text)
    right_signature = self.signature_from_text(right_text)
    estimated = _signature_similarity(left_signature, right_signature)
    exact = self.exact_jaccard(left_text, right_text)
    return PairSimilarityResult(
        left_index=0,
        right_index=1,
        estimated_jaccard=float(estimated),
        exact_jaccard=float(exact),
    )
```

Compare Against Exact Jaccard

- Use exact jaccard as similarity baseline
- Compare minash estimates to true similarity

Runtime Evaluation

```python
num_iterations = 100
sample_size = 20

jaccard_times = []
minhash_times = []
speedups = []

for iteration in range(num_iterations):
    iteration_sample = dataset.sample(sample_size, random_state=42 + iteration)
    texts = iteration_sample["main_content"].fillna("").astype(str).tolist()

    start_time = time.time()
    for i in range(len(texts)):
        for j in range(i + 1, len(texts)):
            _ = minhash.exact_jaccard(texts[i], texts[j])
    jaccard_time = time.time() - start_time
    jaccard_times.append(jaccard_time)

    start_time = time.time()
    _ = minhash.find_candidate_pairs_in_dataframe(
        iteration_sample,
        text_column="main_content",
        id_column="id",
        top_k=5,
        exact_check=False,
    )
    minhash_time = time.time() - start_time
    minhash_times.append(minhash_time)

    if minhash_time > 0:
        speedups.append(jaccard_time / minhash_time)

avg_jaccard_time = sum(jaccard_times) / len(jaccard_times) if jaccard_times else 0.0
avg_minhash_time = sum(minhash_times) / len(minhash_times) if minhash_times else 0.0

print(f"Exact Jaccard average runtime ({num_iterations} runs): {avg_jaccard_time:.4f} seconds")

print("\n========================")
print("MinHash Runtime Analysis")
print("========================")

print(f"MinHash average runtime ({num_iterations} runs): {avg_minhash_time:.4f} seconds")

print("\n========================")
print("Speed Comparison")
print("========================")

if speedups:
    avg_speedup = sum(speedups) / len(speedups)
    variance = sum((value - avg_speedup) ** 2 for value in speedups) / len(speedups)
    std_speedup = variance ** 0.5

    print(f"Average Speedup (Jaccard / MinHash): {avg_speedup:.2f}x")
    print(f"Speedup Standard Deviation: {std_speedup:.2f}x")
else:
    print("Unable to compute speedup (MinHash runtime was zero in all runs).")
```

Evaluate Runtime and Error

- Measure average runtime and speedup
- Analyze approximation error across runs

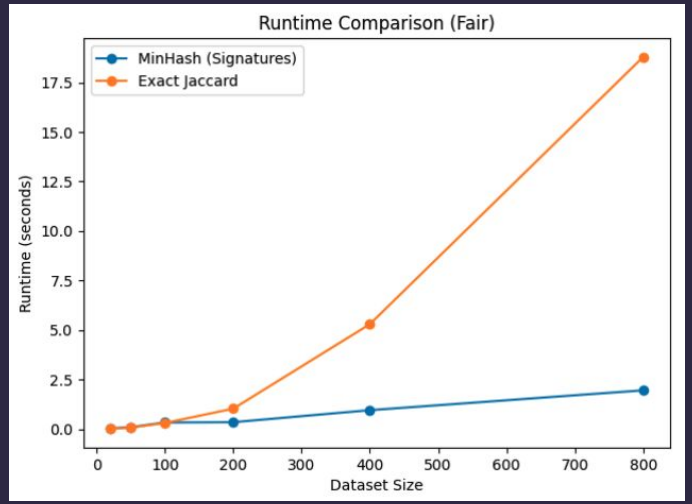
Similarity Evaluation

```python
categories = dataset["category"].dropna().unique()

for cat in categories:
    full_name = category_map.get(cat, "Unknown")
    print(f"\nCategory: {full_name} ({cat})")

    # Take a small sample per category
    cat_df = dataset[dataset["category"] == cat].head(10)

    if len(cat_df) < 2:
        print("Not enough articles.")
        continue

    stats = _pairwise_jaccard_stats(minhash, cat_df)

    if stats:
        max_left, max_right = stats["pair"]
        left_row = cat_df.iloc[max_left]
        right_row = cat_df.iloc[max_right]

        print(f"Average Jaccard Similarity: {stats['average']:.4f}")
        print(f"Max Jaccard Similarity: {stats['maximum']:.4f}")
        print(
            "Max Pair: "
            f"({left_row['id']}, {_article_link(left_row)}) <-> "
            f"({right_row['id']}, {_article_link(right_row)})"
        )
    else:
        print("No valid comparisons.")
```

Category Similarity

- Compare similarity patterns within categories
- Examine average and max overlap

Results and Visualizations: Runtime Comparison



For dataset size 20 with 100 runs:

Exact Jaccard runtime: 0.3016 sec

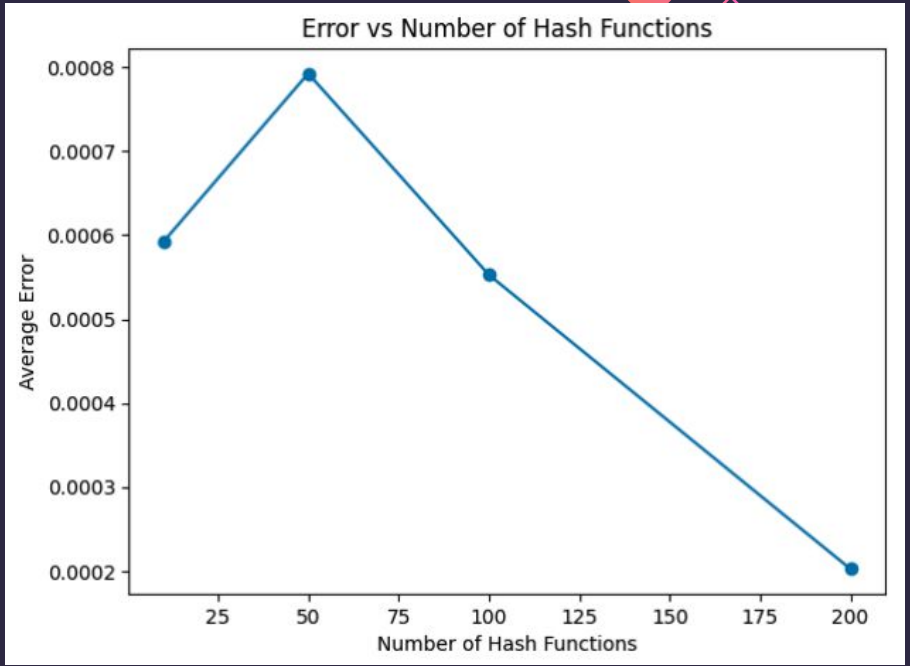
Minhash Runtime: 0.0447 sec

~ 11.22 x speedup

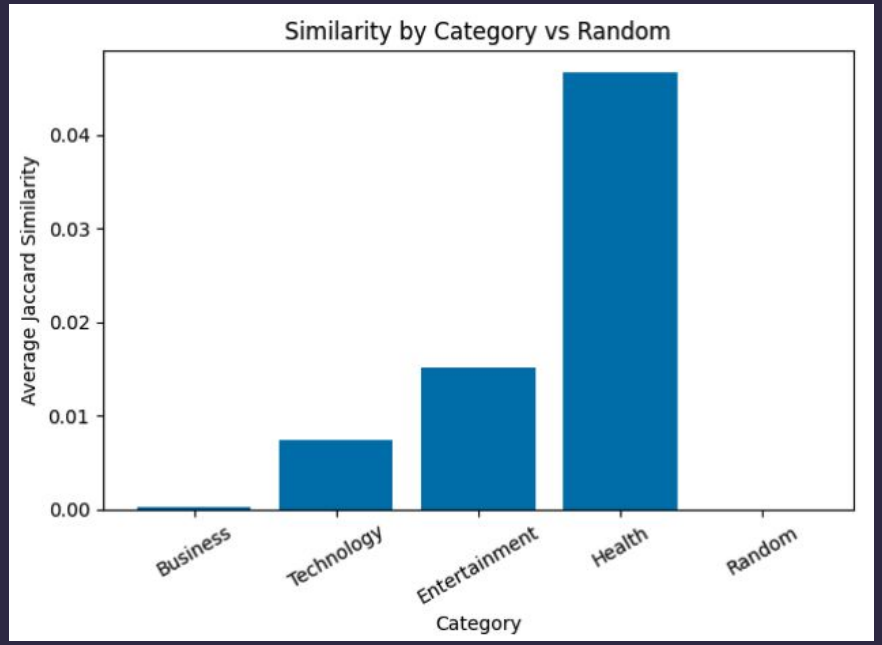
Results and Visualizations: Error

100 Hash Functions → 0.0005 Error

200 Hash Functions → 0.0002 Error



Results and Visualizations: Similarity by Category



Health:

- Avg Jaccard Similarity: 0.0467
- Max Similarity: 0.84

Business:

- Avg Jaccard Similarity: 0.0003
- Max Similarity: 0.0054

Conclusions, Limitations, and Future Work

- Numba-optimized MinHash implementation has a linear-scaling runtime compared to the quadratic-scaling runtime of computing the exact Jaccard similarity with minimal approximation error.
- Non-uniform similarity across different categories, where Health articles shows significantly higher content overlap, suggesting more reliance on standardized terminology or possible plagiarism.
- Implementing Locality-Sensitive Hashing (LSH) to reduce redundant comparisons
- Use clustering models like k-means or GMMs to investigate plagiarism in sub-categories.